

SCAN-PAY-EAT PLATFORM

Complete API, Database Schema & Workflow Technical Specification

Version 1.0.0 | PostgreSQL (Aiven) + Prisma + Express + TypeScript

1. System Overview & Technology Stack

Scan-Pay-Eat is a high-performance, multi-tenant QR food ordering backend designed for food courts, restaurants, and cloud kitchens. The platform guarantees strict shop data isolation, server-side anti-tampering price snapshots, atomic daily token sequence counters, and real-time dashboard order alerts.

Layer	Technology	Purpose & Operational Role
Runtime & Language	Node.js (v26) + TypeScript (v5.7)	Strictly typed asynchronous server with NodeNext module resolution.
Web Framework	Express.js (v4.21)	High-throughput REST API with central error handling & middleware pipelines.
Database & ORM	Aiven PostgreSQL + Prisma ORM (v6)	Fully relational multi-tenant database with connection pool management.
Security & Crypto	Bcrypt + JSON Web Tokens (JWT)	Hashed credentials with rotating refresh tokens in httpOnly SameSite cookies.
Payment Gateway	Razorpay SDK + HMAC-SHA256	Server-side payment order generation and cryptographic signature audits.
Real-Time Gateway	Socket.io (v4.8)	Pub/sub event broadcasting to shopkeeper dashboards & live customer screens.
Validation Engine	Zod (v3.24)	Strict runtime type checking of headers, params, query, and request payloads.
Media Storage	Cloudinary SDK + Multer	Direct cloud image uploads for products with memory buffer streaming.

2. Platform User Roles & Security Matrix

Role	Access Scope	Key Permissions & Boundaries
ADMIN	Platform-Wide	Creates and provisions shops, manages shopkeepers, inspects platform revenue, audits all orders & transactions.
SHOPKEEPER	Shop-Scoped (shopId)	Manages shop categories, products, inventory availability, images, and moves order statuses (CONFIRMED -> COMPLETED).
CUSTOMER	Global Account	Single login across all shops. Can order from any restaurant, view order items, and track live status.

CRITICAL MULTI-TENANCY SECURITY INVARIANT

Every shop-owned entity (Product, Category, Order, Payment, Shopkeeper) is linked by shopId. Shopkeepers can NEVER pass shopId in request bodies; the backend extracts shopId directly from the verified JWT session and injects req.shopId, preventing all horizontal cross-tenant access.

DATABASE SCHEMA SPECIFICATIONS (PRISMA & POSTGRESQL)

The database is engineered in PostgreSQL with strict foreign key constraints, composite unique indexes for idempotency, and decimal precision for financial integrity.

Model: Shop (Table: shops)

Represents independent restaurant tenants operating on subdomains or slugs (e.g., abc.scanpayeat.com).

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Unique shop surrogate identifier.
name	String	NOT NULL	Public trading name of the restaurant.
slug	String	UNIQUE, INDEX	URL slug used for subdomain routing (e.g. 'abc').
subdomain	String	UNIQUE, INDEX	Canonical subdomain for customer QR links.
address	String?	NULLABLE	Physical address for receipt & customer display.
phone	String?	NULLABLE	Direct contact phone number.
qrUrl	String?	NULLABLE	Generated QR ordering URL (https://{subdomain}.scanpayeat.com).
isActive	Boolean	DEFAULT true	Controls tenant accessibility. If false, shopkeeper login & ordering are rejected.
createdAt	DateTime	DEFAULT now()	Timestamp of initial creation.
updatedAt	DateTime	UPDATED AT	Timestamp of most recent modification.

Model: Shopkeeper (Table: shopkeepers)

Staff account credential bound to a single shopId. Must be provisioned by an Admin.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Unique shopkeeper identifier.
shopId	Int	FOREIGN KEY (shops.id)	Mandatory tenant reference. Cannot be modified by shopkeeper.
name	String	NOT NULL	Manager or staff member name.
email	String	UNIQUE, INDEX	Unique login email address.
mobile	String?	NULLABLE	Contact mobile number.
passwordHash	String	NOT NULL	Bcrypt hashed password (10 salt rounds). Plaintext is never stored.
isActive	Boolean	DEFAULT true	Deactivatable by Admin to immediately revoke access.

Model: Customer (Table: customers)

Global consumer account capable of ordering from any restaurant on the platform.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Unique customer identifier.
name	String	NOT NULL	Customer display name.
email	String	UNIQUE, INDEX	Customer login email.

mobile	String?	NULLABLE	10-digit mobile number for order notifications.
passwordHash	String	NOT NULL	Bcrypt password hash.
isActive	Boolean	DEFAULT true	Account active flag.

DATABASE SCHEMA SPECIFICATIONS (PRISMA & POSTGRESQL)

Model: Category (Table: categories)

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Unique category identifier.
shopId	Int	FOREIGN KEY (shops.id)	Shop scope. Composite unique on [shopId, name].
name	String	NOT NULL	Category title (e.g., 'Burgers', 'Beverages').
description	String?	NULLABLE	Brief category overview.
sortOrder	Int	DEFAULT 0	Ordering priority in public menu display.
isActive	Boolean	DEFAULT true	Toggle to show or hide category.

Model: Product (Table: products)

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Unique product identifier.
shopId	Int	FOREIGN KEY (shops.id)	Shop scope. Enforces tenant boundary.
categoryId	Int	FOREIGN KEY (categories.id)	Parent category. Must belong to the same shopId.
name	String	NOT NULL	Food item name (e.g. 'Chicken Burger').
description	String?	NULLABLE	Item ingredients & culinary description.
price	Decimal(10,2)	NOT NULL	Current live retail price in INR.
imageUrl	String?	NULLABLE	Cloudinary CDN image URL.
isAvailable	Boolean	DEFAULT true	Live availability toggle (true = In Stock, false = Sold Out).

Model: Order (Table: orders)

Central transactional entity linking customer, items, and status transitions.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Order internal surrogate key.
orderCode	String	UNIQUE, INDEX	Human-readable order tracking code (e.g. ORD1790954944).
shopId	Int	FOREIGN KEY (shops.id)	Restaurant where the order was placed.
customerId	Int?	NULLABLE, FOREIGN KEY	Customer account reference (or null for guest checkout).
tokenNumber	String?	NULLABLE, INDEX	Daily shop sequential token (e.g. A101, A102). Assigned upon payment.
orderStatus	OrderStatus	ENUM	PENDING CONFIRMED PREPARING READY COMPLETED CANCELLED.
paymentStatus	PaymentStatus	ENUM	PENDING PAID FAILED REFUNDED.
subtotal	Decimal(10,2)	NOT NULL	Sum of all order items calculated strictly by backend from DB.
tax	Decimal(10,2)	DEFAULT 0.00	Tax amount.

totalAmount	Decimal(10,2)	NOT NULL	Total payable amount (subtotal + tax).
notes	String?	NULLABLE	Customer preparation instructions (e.g. 'No onions').

Model: OrderItem (Table: order_items) -- Historical Snapshot

Immutable price and name snapshot. If product prices change later, past order receipts remain exact.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Order item line identifier.
orderId	Int	FOREIGN KEY (orders.id)	Parent order reference with CASCADE on delete.
productId	Int	FOREIGN KEY (products.id)	Original product catalog reference.
productName	String	NOT NULL	Snapshotted product name at time of checkout.
unitPrice	Decimal(10,2)	NOT NULL	Snapshotted unit price at time of purchase.
quantity	Int	NOT NULL, >= 1	Purchased count.
lineTotal	Decimal(10,2)	NOT NULL	Computed as unitPrice * quantity.

DATABASE SCHEMA SPECIFICATIONS (PRISMA & POSTGRESQL)

Model: Payment (Table: payments)

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Internal payment ID.
orderId	Int	FOREIGN KEY (orders.id)	Associated order.
shopId	Int	FOREIGN KEY (shops.id)	Recipient shop ID.
customerId	Int?	NULLABLE	Payer customer ID.
amount	Decimal(10,2)	NOT NULL	Authorized charge amount in INR.
currency	String	DEFAULT 'INR'	Transaction currency.
provider	String	DEFAULT 'RAZORPAY'	Payment gateway provider.
razorpayOrderId	String?	UNIQUE, INDEX	Gateway order ID returned by Razorpay API.
razorpayPaymentId	String?	UNIQUE, INDEX	Capture ID returned upon successful customer authorization.
razorpaySignature	String?	NULLABLE	HMAC-SHA256 signature generated by Razorpay.
status	PaymentStatus	ENUM	PENDING PAID FAILED REFUNDED.

Model: TokenCounter (Table: token_counters) -- Concurrency Protection

Maintains daily per-shop sequence counters. Prevents duplicate token collisions during high-concurrency checkout.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Counter surrogate key.
shopId	Int	FOREIGN KEY (shops.id)	Shop scope. Composite unique on [shopId, date].
date	String	NOT NULL	Calendar date in 'YYYY-MM-DD' format (UTC/IST).
lastSequence	Int	DEFAULT 100	Last assigned sequential integer (starts at 101 each day).
updatedAt	DateTime	UPDATED AT	Timestamp of last sequence increment.

Model: OrderStatusHistory (Table: order_status_history)

Immutable audit trail tracking every lifecycle transition for an order with actor attribution.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	History log ID.
orderId	Int	FOREIGN KEY (orders.id)	Parent order reference.
status	OrderStatus	ENUM	Order state (CONFIRMED, PREPARING, READY, etc.).
notes	String?	NULLABLE	Reason or descriptive kitchen remarks.
changedByRole	UserRole?	NULLABLE	Role who triggered change (ADMIN, SHOPKEEPER, SYSTEM).
changedById	Int?	NULLABLE	User ID of the actor.
createdAt	DateTime	DEFAULT now()	Timestamp when state transition took effect.

Model: PaymentWebhookEvent (Table: payment_webhook_events)

Guarantees idempotent webhook processing. Ensures Razorpay duplicate delivery does not re-process orders.

Field	Type	Constraints	Description
id	Int	PRIMARY KEY, AUTO	Internal event ID.
eventId	String	UNIQUE, INDEX	Razorpay event identifier (e.g. 'evt_100245').
eventType	String	NOT NULL	Event name (e.g., 'payment.captured', 'payment.failed').
payload	Json	NOT NULL	Raw JSON payload delivered by gateway.
status	String	DEFAULT 'PROCESSED'	Processing disposition.
processedAt	DateTime	DEFAULT now()	Time recorded.

HOW EACH FLOW WORKS (END-TO-END TECHNICAL EXECUTION)

Flow 1: Customer Registration & Unified Authentication

1. Registration (POST /api/auth/register): The client submits name, email, mobile, and password. Zod validates the schema. The backend verifies email uniqueness, hashes the password via Bcrypt (10 salt rounds), creates the Customer in PostgreSQL, and generates an access token (15m expiry) and refresh token (7d expiry).
2. Token Storage: The SHA-256 hash of the refresh token is stored in the refresh_tokens table. Both tokens are delivered to the client as secure, httpOnly, SameSite cookies with an optional Bearer header for mobile apps.
3. Unified Login (POST /api/auth/login): Single login endpoint for all three roles (Admin, Shopkeeper, Customer). The backend checks the Admin table, then Shopkeeper (verifying shop.isActive), then Customer. Once password verification passes, claims are bundled: { sub: userId, role, email, shopId }.
4. Token Rotation (POST /api/auth/refresh): When an access token expires, the client sends the refresh token. The backend verifies signature, checks DB for active/non-revoked status, revokes the old token, and issues a fresh pair. If an already revoked token is used, all tokens for that user are revoked (replay attack defense).

Flow 2: Admin Multi-Tenant Provisioning

1. Shop Creation (POST /api/admin/shops): Admin creates a shop by providing name, slug, subdomain, address, and phone. The backend enforces alphanumeric/hyphen slug uniqueness and automatically constructs the public QR URL (https://{subdomain}.scanpayeat.com).
2. Shopkeeper Creation (POST /api/admin/shopkeepers): Admin provisions a shopkeeper, assigning them to a specific shopId. Public registration for shopkeepers is disabled.
3. Tenant Kill-Switch (PATCH /api/admin/shops/:id/status): When an Admin marks a shop inactive, all subsequent shopkeeper logins and customer menu access for that shop are blocked immediately.

Flow 3: Shopkeeper Menu & Multi-Tenant Scoping

1. Zero-Trust shopId Rule: All requests to /api/shop/* execute authenticate and shopScope middlewares. The backend strictly resolves req.shopId = req.user.shopId from the verified JWT. Frontend-submitted shopId values are completely ignored.
2. Category & Product CRUD: Shopkeepers create categories and products. When creating a product, the backend checks that the referenced categoryId belongs to req.shopId, preventing foreign category contamination.
3. Availability & Images: Shopkeepers can toggle products available/sold-out (PATCH /api/shop/products/:id/availability) or upload images (POST /api/shop/upload) via Multer buffer stream to Cloudinary.

Flow 4: Public QR Menu Resolution

1. QR Scan: When a diner scans a table QR code, their browser navigates to https://abc.scanpayeat.com.
2. Slug Resolution: The Next.js frontend calls GET /api/public/shops/abc and GET /api/public/shops/abc/menu.
3. Public Filtering: No login is required. The backend checks shop.isActive, returning categories and only products where isAvailable == true.

HOW EACH FLOW WORKS (CHECKOUT, PAYMENTS & TOKENS)

Flow 5: Anti-Tampering Order Checkout (POST /api/orders/checkout)

1. Payload: Client submits only product IDs and quantities: { shopSlug: 'abc', items: [{ productId: 10, quantity: 2 }] }.
2. Server Price Recalculation: The backend queries PostgreSQL for all requested product IDs within that shop. If any product is missing, belongs to another shop, or is marked unavailable, checkout is rejected with 400 Bad Request.
3. Snapshot Generation: Database prices are read directly: $\text{lineTotal} = \text{unitPrice} * \text{quantity}$; $\text{subtotal} = \text{sum}(\text{lineTotals})$.
4. Pending Order: In a single database transaction, the backend creates an Order (status PENDING, paymentStatus PENDING), inserts OrderItem snapshot rows (productName, unitPrice, quantity, lineTotal), creates a Razorpay gateway order, and creates a Payment record (status PENDING).
5. Gateway Response: Returns { orderId, orderCode, totalAmount, razorpayOrderId, currency: 'INR', key: RAZORPAY_KEY_ID } to the frontend to initialize checkout.

Flow 6: Razorpay Payment Verification & Atomic Transaction

1. Customer Pays: Customer completes payment on Razorpay checkout modal. Gateway returns razorpay_order_id, razorpay_payment_id, and razorpay_signature.
2. Signature Audit: Client posts tokens to POST /api/payments/verify. Backend computes $\text{HMAC_SHA256}(\text{order_id} + '|' + \text{payment_id}, \text{RAZORPAY_KEY_SECRET})$ and uses crypto.timingSafeEqual to verify the signature.
3. Atomic Transaction: If valid, the backend executes an atomic PostgreSQL transaction with a 15-second pool timeout:

ATOMIC PAYMENT COMMITMENT SEQUENCE

1. Mark Payment status = PAID with razorpayPaymentId and signature.
 2. Atomically upsert & increment TokenCounter for (shopId, date) -> yields tokenNumber (e.g. A101).
 3. Update Order: paymentStatus = PAID, orderStatus = CONFIRMED, tokenNumber = 'A101'.
 4. Insert OrderStatusHistory log: status = CONFIRMED, notes = 'Payment verified via Razorpay'.
 5. COMMIT transaction.
4. Real-Time Alert: The backend emits a 'new_order' event over Socket.io to room shop:{shopId} containing the order code, token number, and items. The shopkeeper's tablet sounds an alert and updates the queue.

Flow 7: Daily Token Counter Concurrency Protection

Every shop has a distinct token sequence (e.g., Shop 'ABC' uses 'A101', 'A102'; Shop 'Burger' uses 'B101', 'B102'). In PostgreSQL, Prisma executes an atomic upsert with { increment: 1 } on the token_counters table with a composite key [shopId, date]. PostgreSQL locks the counter row for the microsecond duration of the update, completely preventing duplicate tokens when multiple customers pay simultaneously.

Flow 8: Webhook Idempotency (POST /api/webhooks/razorpay)

1. Raw Body Signature Audit: Preserves raw request bytes and verifies x-razorpay-signature with RAZORPAY_WEBHOOK_SECRET.
2. Deduplication: Checks payment_webhook_events table for eventId. If already present, immediately returns 200 { status: 'already_processed' }.
3. Event Handling: On payment.captured, executes the same atomic confirmation transaction if the order was not already confirmed. On payment.failed, marks Payment and Order as FAILED.

HOW EACH FLOW WORKS & API REFERENCE DIRECTORY

Flow 9: Shopkeeper Status Flow & Socket.io Realtime

1. Order Lifecycle: Orders move sequentially: CONFIRMED -> PREPARING -> READY -> COMPLETED (or CANCELLED).
2. PATCH /api/shop/orders/:id/status: Validates that the order belongs to req.shopId, updates orderStatus, logs an OrderStatusHistory record with actor ID and timestamp, and broadcasts 'order_status_updated' to shop:{shopId} and order:{orderId}.

Flow 10: Customer Order Tracking & History

1. Global Order History (GET /api/me/orders): Filters strictly by authenticated customerId = req.user.id. Customers see orders from all visited shops.
2. Real-Time Tracking (GET /api/me/orders/:orderCode): Customer order screen subscribes to Socket.io room order:{orderId} to receive live status updates without refreshing.

Complete API Endpoint Reference Directory

Method	Endpoint Route	Auth / Roles	Summary & Response
POST	/api/auth/register	Public	Registers customer, sets httpOnly cookies. Returns user.
POST	/api/auth/login	Public	Unified login (Admin/Shopkeeper/Customer). Sets cookies.
POST	/api/auth/refresh	Public (Cookie)	Rotates refresh token and returns new access token.
POST	/api/auth/logout	Public	Revokes refresh token in DB and clears cookies.
GET	/api/auth/me	All Roles	Returns fresh authenticated user profile and permissions.
POST	/api/admin/shops	ADMIN	Creates shop with slug/subdomain and generates QR URL.
GET	/api/admin/shops	ADMIN	Lists all shops with search, active filters, and pagination.
GET	/api/admin/shops/:id	ADMIN	Fetches single shop details with staff counts.
PUT	/api/admin/shops/:id	ADMIN	Updates shop name, slug, address, or phone.
PATCH	/api/admin/shops/:id/status	ADMIN	Activates/deactivates shop (disables staff logins).
POST	/api/admin/shopkeepers	ADMIN	Provisions a shopkeeper account linked to a shopId.
GET	/api/admin/shopkeepers	ADMIN	Lists shopkeepers with shopId filter and pagination.
GET	/api/admin/dashboard	ADMIN	Aggregates total shops, staff, customers, orders, revenue.
GET	/api/admin/orders	ADMIN	Global filterable order listing across all restaurants.
GET	/api/admin/transactions	ADMIN	Global payment audit logs with status filters.
POST	/api/shop/categories	SHOPKEEPER	Creates category scoped strictly to req.shopId.

GET	/api/shop/categories	SHOPKEEPER	Lists all categories for the authenticated shop.
POST	/api/shop/products	SHOPKEEPER	Creates product with price, description, and image.
GET	/api/shop/products	SHOPKEEPER	Lists shop products with category & availability filters.
PATCH	/api/shop/products/:id/availability	SHOPKEEPER	Toggles product available (true) or sold-out (false).
POST	/api/shop/upload	SHOPKEEPER	Uploads food photo via Multer stream to Cloudinary.
GET	/api/shop/orders	SHOPKEEPER	Lists incoming orders for the authenticated shop.
PATCH	/api/shop/orders/:id/status	SHOPKEEPER	Advances order status (CONFIRMED -> COMPLETED).
GET	/api/public/shops/:slug	Public	Resolves shop metadata by subdomain or slug.
GET	/api/public/shops/:slug/menu	Public	Returns live categories and available products.
POST	/api/orders/checkout	Public / Cust	Validates DB prices, creates pending order & Razorpay ID.
POST	/api/payments/verify	Public / Cust	Verifies HMAC signature, assigns daily token atomically.
POST	/api/webhooks/razorpay	Public (Gateway)	Processes payment.captured & failed with idempotency.
GET	/api/me/orders	CUSTOMER	Customer order history across all restaurants.
GET	/api/me/orders/:orderCode	CUSTOMER	Fetches single order details and lifecycle status history.
GET	/api-docs	Public	Interactive Swagger UI documentation page.
GET	/api-docs/openapi.json	Public	Raw OpenAPI 3.0 JSON specification.

SWAGGER UI GUIDE & SEED CREDENTIALS

1. Interactive Swagger UI Access

The backend embeds an interactive OpenAPI 3.0 Swagger UI interface:

SWAGGER UI ENDPOINT

URL: <http://localhost:5001/api-docs> OpenAPI Spec: <http://localhost:5001/api-docs/openapi.json> You can execute live API requests directly from Swagger UI by clicking 'Try it out'. For protected routes, obtain a token from POST [/api/auth/login](#) and click the 'Authorize' button in Swagger UI.

2. Seeded Test Accounts & Demo Credentials

Run 'npm run seed' to initialize the database with standard demo accounts:

Account Type	Email / Identifier	Password	Permissions & Assigned Tenant
Platform Admin	admin@scanpayeat.com	Admin@123	Full access to /api/admin/* dashboard and management.
Shopkeeper	shop@abc.com	Shop@123	Managing 'ABC Restaurant' (Shop ID: 1, Slug: 'abc').
Customer	customer@demo.com	Customer@123	Customer ordering account with access to /api/me/* .
Demo Shop Slug	abc	N/A	Public menu at /api/public/shops/abc/menu .

3. Production Deployment & Readiness Checklist

Security Control	Status	Implementation Detail
Anti-Tampering Prices	ACTIVE	Prices recalculated strictly from PostgreSQL database.
Multi-Tenant Isolation	ACTIVE	req.shopId enforced via JWT middleware on every shop query.
Concurrency Protection	ACTIVE	TokenCounter atomic upsert prevents duplicate order tokens.
Signature Verification	ACTIVE	Cryptographic HMAC-SHA256 verification on payments & webhooks.
Webhook Idempotency	ACTIVE	payment_webhook_events table blocks duplicate processing.
Rate Limiting	ACTIVE	Express rate limiters on auth, checkout, and payment routes.
Strict Input Validation	ACTIVE	Zod schemas validate all body, query, and path parameters.
CORS & Cookie Security	ACTIVE	HttpOnly SameSite cookies with credentials enabled.